УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 19

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С БИБЛИОТЕКОЙ REQUESTS И МОДУЛЕМ URLLIB.REQUEST

ЛАБОРАТОРНАЯ РАБОТА № 19.1

СОЗДАНИЕ ПРОСТЕЙШИХ ПРОГРАММ ДЛЯ РАБОТЫ С БИБЛИОТЕКОЙ REQUESTS

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейших программ для работы с библиотекой Requests

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Введение в библиотеку Requests

Requests — это простая и мощная библиотека для работы с HTTP-запросами в Python. Она обеспечивает удобный интерфейс для выполнения операций с REST API, загрузки данных с веб-страниц, а также отправки файлов и авторизации. Благодаря лаконичному синтаксису и широкому функционалу, Requests стала стандартом де-факто среди HTTP-клиентов Python.

Библиотека Requests была создана Кеннетом Рейтцем в 2011 году с целью сделать HTTP-запросы в Python максимально простыми и удобными. Она построена поверх urllib3 и предоставляет высокоуровневый API для работы с HTTP-протоколом.

Преимущества Requests по сравнению с urllib:

- более читаемый синтаксис — интуитивно понятные методы и параметры;
- автоматическая обработка куки и редиректов — не требует дополнительной настройки;
- удобная работа с JSON, заголовками, формами — встроенная поддержка популярных форматов;
- поддержка сессий, прокси, аутентификации — полный набор инструментов для работы с веб-сервисами;
- поддержка всех методов HTTP — GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS;
- автоматическое декодирование контента — интеллектуальная обработка кодировок;
- встроенная поддержка SSL — безопасные соединения по умолчанию.
-

4.2 Установка и подключение библиотеки

Установка с помощью pip:
pip install requests
Для обновления до последней версии:
pip install --upgrade requests
Импорт библиотеки:
import requests

4.3 Основные методы HTTP-запросов

Отправка GET-запросов
GET-запросы используются для получения данных с сервера:
response = requests.get('https://api.example.com/data')
print(response.status_code) # Код ответа
print(response.text) # Текст ответа
Получение JSON-данных:
response = requests.get('https://api.example.com/users')
data = response.json() # Автоматическое преобразование в словарь Python
print(data)
Отправка POST-запросов
POST-запросы используются для отправки данных на сервер:
payload = {'username': 'admin', 'password': '123'}
response = requests.post('https://api.example.com/login', data=payload)
Отправка JSON-данных:
user_data = {'name': 'John', 'email': 'john@example.com'}
response = requests.post('https://api.example.com/users', json=user_data)
Использование PUT, PATCH и DELETE
PUT - полное обновление ресурса
requests.put('https://api.example.com/user/1', json={'name': 'New Name', 'email': 'new@example.com'})
PATCH - частичное обновление ресурса
requests.patch('https://api.example.com/user/1', json={'active': True})
DELETE - удаление ресурса
requests.delete('https://api.example.com/user/1')
HEAD - получение только заголовков
response = requests.head('https://api.example.com/user/1')
print(response.headers)

4.4 Работа с параметрами запроса

Параметры URL можно передавать через словарь:
params = {'search': 'python', 'limit': 10, 'page': 1}
response = requests.get('https://api.example.com/search', params=params)
print(response.url) # Покажет полный URL с параметрами

Можно также передавать параметры в виде списка кортежей:

```
params = [('tag', 'python'), ('tag', 'web'), ('limit', 20)]
response = requests.get('https://api.example.com/posts', params=params)
```

4.5 Работа с заголовками и куки

Заголовки запроса

```
headers = {
    'Authorization': 'Bearer TOKEN123',
    'User-Agent': 'MyApp/1.0',
    'Content-Type': 'application/json'
}
response = requests.get('https://api.example.com/data', headers=headers)
```

Заголовки ответа

```
response = requests.get('https://api.example.com/data')
print(response.headers['Content-Type'])
print(response.headers.get('Server', 'Unknown'))
```

Работа с куки

```
# Отправка куки
cookies = {'sessionid': 'abc123', 'csrftoken': 'xyz789'}
response = requests.get('https://example.com', cookies=cookies)
# Получение куки из ответа
response = requests.get('https://example.com')
print(response.cookies)
for cookie in response.cookies:
    print(f'{cookie.name}: {cookie.value}')
```

4.6 Отправка и загрузка файлов

Отправка файлов

```
# Отправка одного файла
files = {'file': open('report.pdf', 'rb')}
response = requests.post('https://api.example.com/upload', files=files)
# Отправка нескольких файлов
files = {
    'file1': open('document.pdf', 'rb'),
    'file2': open('image.png', 'rb')
}
response = requests.post('https://api.example.com/upload', files=files)
# Отправка файла с дополнительными данными
files = {'file': ('report.pdf', open('report.pdf', 'rb'), 'application/pdf')}
data = {'description': 'Monthly report'}
response = requests.post('https://api.example.com/upload', files=files,
data=data)
```

Загрузка файлов

```
# Загрузка файла в память
response = requests.get('https://example.com/file.zip')
with open('output.zip', 'wb') as f:
    f.write(response.content)
# Потоковая загрузка больших файлов
response = requests.get('https://example.com/large-file.zip', stream=True)
with open('large-file.zip', 'wb') as f:
    for chunk in response.iter_content(chunk_size=8192):
        f.write(chunk)
```

4.7 Обработка JSON и форм

Работа с JSON

```
# Отправка JSON
data = {'name': 'Alice', 'age': 30}
response = requests.post('https://api.example.com/users', json=data)
# Получение JSON
response = requests.get('https://api.example.com/users/1')
try:
    user_data = response.json()
    print(user_data['name'])
except ValueError:
    print("Ответ не содержит валидный JSON")
```

Отправка форм

```
# Форма с данными
form_data = {
    'username': 'user123',
    'password': 'secret',
    'remember': 'on'
}
response = requests.post('https://example.com/login', data=form_data)
# Форма с файлом
form_data = {'title': 'My Document'}
files = {'document': open('doc.pdf', 'rb')}
response = requests.post('https://example.com/upload', data=form_data,
files=files)
```

4.8 Работа с сессиями

Сессии позволяют сохранять состояние между запросами:

```
session = requests.Session()
# Настройка сессии
session.headers.update({'User-Agent': 'MyApp/1.0'})
session.auth = ('username', 'password')
```

```
# Выполнение запросов через сессию
session.get('https://example.com/login')
response = session.post('https://example.com/dashboard', data={'key': 'value'})
response = session.get('https://example.com/profile')
# Закрытие сессии
session.close()
```

Использование сессии в контекстном менеджере:

```
with requests.Session() as session:
    session.get('https://example.com/login')
    response = session.get('https://example.com/data')
```

4.9 Обработка исключений и ошибок

```
try:
    response = requests.get('https://api.example.com/data', timeout=5)
    response.raise_for_status()  # Вызывает исключение для 4xx и 5xx статусов
    data = response.json()
except requests.exceptions.HTTPError as e:
    print(f"HTTP ошибка: {e}")
except requests.exceptions.ConnectionError as e:
    print(f"Ошибка подключения: {e}")
except requests.exceptions.Timeout as e:
    print(f"Таймаут: {e}")
except requests.exceptions.RequestException as e:
    print(f"Общая ошибка запроса: {e}")
except ValueError as e:
    print(f"Ошибка парсинга JSON: {e}")
```

4.10 Установка таймаутов и повторных попыток

Таймауты

```
# Общий таймаут
response = requests.get('https://example.com', timeout=5)
# Раздельные таймауты для подключения и чтения
response = requests.get('https://example.com', timeout=(3, 10))
```

Повторные попытки

```
from requests.adapters import HTTPAdapter
from requests.packages.urllib3.util.retry import Retry
session = requests.Session()
# Настройка стратегии повторов
retry_strategy = Retry(
    total=3,                # Общее количество попыток
    backoff_factor=1,       # Множитель для экспоненциального backoff
    status_forcelist=[429, 500, 502, 503, 504],  # Коды для повтора
```

```
    method_whitelist=["HEAD", "GET", "OPTIONS"]   # Методы для повтора
)
adapter = HTTPAdapter(max_retries=retry_strategy)
session.mount("http://", adapter)
session.mount("https://", adapter)
response = session.get("https://example.com")
```

4.11 Аутентификация

Basic Authentication

```
from requests.auth import HTTPBasicAuth
# Способ 1
response = requests.get('https://api.example.com/data',
                    auth=HTTPBasicAuth('username', 'password'))
# Способ 2 (сокращенный)
response = requests.get('https://api.example.com/data',
                    auth=('username', 'password'))
```

Token Authentication

```
# Bearer Token
headers = {'Authorization': 'Bearer your_access_token'}
response = requests.get('https://api.example.com/secure', headers=headers)
# API Key
headers = {'X-API-Key': 'your_api_key'}
response = requests.get('https://api.example.com/data', headers=headers)
```

Digest Authentication

```
from requests.auth import HTTPDigestAuth

response = requests.get('https://api.example.com/data',
                    auth=HTTPDigestAuth('username', 'password'))
```

4.12 Прокси и SSL-сертификаты

Использование прокси

```
proxies = {
   'http': 'http://10.10.1.10:3128',
   'https': 'http://10.10.1.10:1080',
}
response = requests.get('https://example.com', proxies=proxies)
# Прокси с аутентификацией
proxies = {
   'http': 'http://user:pass@10.10.1.10:3128',
   'https': 'http://user:pass@10.10.1.10:1080',
}
```

SSL-сертификаты

```
# Отключение проверки SSL (не рекомендуется в продакшене)
```

```
response = requests.get('https://self-signed.badssl.com/', verify=False)
# Указание пути к сертификату
response = requests.get('https://example.com', verify='/path/to/cert.pem')
# Указание сертификата клиента
response = requests.get('https://example.com',
                        cert=('/path/to/client.cert', '/path/to/client.key'))
```

4.13 Логирование и отладка запросов

Включение логирования

```
import logging
import requests
# Включение подробного логирования
logging.basicConfig(level=logging.DEBUG)
# Настройка логирования только для requests
requests_log = logging.getLogger("requests.packages.urllib3")
requests_log.setLevel(logging.DEBUG)
requests_log.propagate = True
response = requests.get('https://example.com')
```

Отладка с помощью hooks

```
def print_request_response(response, *args, **kwargs):
    print(f"Запрос: {response.request.method} {response.request.url}")
    print(f"Ответ: {response.status_code}")
response = requests.get('https://example.com', hooks={'response':
print_request_response})
```

4.14 Работа с потоками данных

Потоковая загрузка

```
response = requests.get('https://example.com/large-file.zip', stream=True)
# Загрузка по частям
with open('large-file.zip', 'wb') as f:
    for chunk in response.iter_content(chunk_size=1024):
        if chunk:
            f.write(chunk)
# Построчная загрузка текстовых файлов
response = requests.get('https://example.com/data.txt', stream=True)
for line in response.iter_lines():
    print(line.decode('utf-8'))
```

Кастомные адаптеры

```
class TimeoutHTTPAdapter(HTTPAdapter):
    def __init__(self, *args, **kwargs):
        self.timeout = kwargs.pop("timeout", 5)
        super().__init__(*args, **kwargs)
    def send(self, request, **kwargs):
```

```
        kwargs["timeout"] = self.timeout
        return super().send(request, **kwargs)
session = requests.Session()
session.mount("https://", TimeoutHTTPAdapter(timeout=10))
```

4.15 Таблица методов и функций библиотеки Requests

Таблица 1 - Методы и функции библиотеки Requests

Метод/Функция	Описание	Пример использования
requests.get()	Выполняет GET-запрос	requests.get('https://api.example.com')
requests.post()	Выполняет POST-запрос	requests.post('https://api.example.com', data={'key': 'value'})
requests.put()	Выполняет PUT-запрос	requests.put('https://api.example.com/1', json={'name': 'New'})
requests.patch()	Выполняет PATCH-запрос	requests.patch('https://api.example.com/1', json={'active': True})
requests.delete()	Выполняет DELETE-запрос	requests.delete('https://api.example.com/1')
requests.head()	Выполняет HEAD-запрос	requests.head('https://api.example.com')
requests.options()	Выполняет OPTIONS-запрос	requests.options('https://api.example.com')
requests.Session()	Создает объект сессии	session = requests.Session()
response.json()	Преобразует ответ в JSON	data = response.json()
response.text	Возвращает текст ответа	content = response.text
response.content	Возвращает бинарное содержимое	binary_data = response.content
response.status_code	Возвращает HTTP статус код	if response.status_code == 200:
response.headers	Возвращает заголовки ответа	content_type = response.headers['Content-Type']
response.cookies	Возвращает куки ответа	session_id = response.cookies['sessionid']
response.url	Возвращает финальный URL	final_url = response.url
response.raise_for_status()	Вызывает исключение для ошибок HTTP	response.raise_for_status()
response.iter_content()	Итерирует по содержимому порциями	for chunk in response.iter_content(1024):
response.iter_lines()	Итерирует по строкам	for line in response.iter_lines():
HTTPBasicAuth()	Базовая HTTP аутентификация	auth=HTTPBasicAuth('user', 'pass')
HTTPDigestAuth()	Digest аутентификация	auth=HTTPDigestAuth('user', 'pass')
HTTPAdapter()	Адаптер для настройки транспорта	adapter = HTTPAdapter(max_retries=3)
Retry()	Конфигурация повторных попыток	retry = Retry(total=3, backoff_factor=1)

4.16 Параметры основных методов

Таблица 2 - Параметры основных методов

Параметр	Описание	Применимость
url	URL для запроса	Все методы
params	Параметры URL	Все методы
data	Данные формы	POST, PUT, PATCH
json	JSON данные	POST, PUT, PATCH
headers	Заголовки запроса	Все методы
cookies	Куки	Все методы
files	Файлы для загрузки	POST, PUT, PATCH
proxies	Прокси серверы	Все методы
verify	Проверка SSL	Все методы
cert	Клиентский сертификат	Все методы
stream	Потоковая загрузка	Все методы
allow_redirects	Разрешить редиректы	Все методы

4.17 Примеры практического применения

Интеграция с внешними API

```
# Работа с GitHub API
def get_user_repos(username):
    url = f'https://api.github.com/users/{username}/repos'
    response = requests.get(url)
    if response.status_code == 200:
        return response.json()
    return None
repos = get_user_repos('octocat')
for repo in repos:
    print(f"{repo['name']}: {repo['description']}")
```

Мониторинг сервисов

```
def check_service_health(url):
    try:
        response = requests.get(url, timeout=5)
        return {
            'status': 'UP' if response.status_code == 200 else 'DOWN',
            'response_time': response.elapsed.total_seconds(),
            'status_code': response.status_code
        }
    except requests.exceptions.RequestException:
        return {'status': 'DOWN', 'response_time': None, 'status_code': None}
health = check_service_health('https://example.com/health')
print(f"Service status: {health['status']}")
```

Отправка уведомлений в Telegram

```
def send_telegram_message(bot_token, chat_id, message):
```

```
    url = f'https://api.telegram.org/bot{bot_token}/sendMessage'
    data = {
        'chat_id': chat_id,
        'text': message,
        'parse_mode': 'HTML'
    }
    response = requests.post(url, data=data)
    return response.json()
# Пример использования
send_telegram_message('YOUR_BOT_TOKEN', 'CHAT_ID', 'Привет из Python!')
```

Скрипты автоматизации

```
def download_and_process_data():
    # Загрузка данных
    response = requests.get('https://api.example.com/data.json')
    data = response.json()
    # Обработка данных
    processed_data = [item for item in data if item['active']]
    # Отправка обработанных данных
    result = requests.post('https://api.example.com/processed', json=processed_data)
    return result.status_code == 200
success = download_and_process_data()
print(f"Обработка {'успешна' if success else 'не удалась'}")
```

4.18 Оптимизация производительности

Повторное использование соединений

```
# Неэффективно - создает новое соединение для каждого запроса
for i in range(10):
    response = requests.get(f'https://api.example.com/data/{i}')
# Эффективно - использует пул соединений
with requests.Session() as session:
    for i in range(10):
        response = session.get(f'https://api.example.com/data/{i}')
```

4.19 Асинхронные альтернативы

Для высокопроизводительных приложений рассмотрите использование:

- aiohttp для асинхронных HTTP-запросов;
- httpx как современная альтернатива с поддержкой async/await;
- grequests для псевдо-асинхронных запросов.

4.20 Безопасность

Валидация SSL-сертификатов

```
# Всегда проверяйте SSL-сертификаты в продакшене
response = requests.get('https://api.example.com', verify=True)
# Для самоподписанных сертификатов указывайте путь
response = requests.get('https://internal-api.company.com',
                        verify='/path/to/ca-bundle.crt')
```

Защита от раскрытия данных

```
# Не логируйте чувствительные данные
import logging
logging.getLogger("requests.packages.urllib3").setLevel(logging.WARNING)
# Используйте переменные окружения для токенов
import os
api_key = os.getenv('API_KEY')
headers = {'Authorization': f'Bearer {api_key}'}
```

4.21 Пример решения задачи на языке программирования Python

Задача. Напишите программу на языке Python с использованием библиотеки Requests, которая отправляет GET-запрос к указанной веб-странице, получает HTTP-ответ от сервера и выводит на экран код состояния ответа, значение заголовка Content-Type, а также количество символов в полученном содержимом страницы.

Решение задачи:

```
import requests
url = input("Введите URL: ")
response = requests.get(url)
# Код состояния
print("Код состояния:", response.status_code)
# Заголовок Content-Type
content_type = response.headers.get("Content-Type")
print("Content-Type:", content_type)
# Количество символов в ответе
print("Количество символов:", len(response.text))
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Напишите программу, которая выполняет GET-запрос к веб-странице и выводит общее количество символов в ответе.

2 Напишите программу, которая отправляет GET-запрос и определяет, был ли запрос успешно выполнен.

3 Напишите программу, которая получает HTTP-ответ и выводит значение заголовка Content-Type.

4 Напишите программу, которая отправляет GET-запрос и выводит первые 5 строк ответа сервера.

5 Напишите программу, которая выполняет GET-запрос и сохраняет HTML-код страницы в файл page.html.

6 Напишите программу, которая отправляет GET-запрос и выводит URL, по которому был выполнен запрос.

7 Напишите программу, которая выполняет запрос к веб-ресурсу с тайм-аутом соединения.

8 Напишите программу, которая получает HTTP-ответ и выводит информацию о кодировке содержимого.

9 Напишите программу, которая отправляет GET-запрос и выводит дату ответа сервера из заголовков.

10 Напишите программу, которая выполняет GET-запрос и определяет, использовалось ли перенаправление.

11 Напишите программу, которая отправляет GET-запрос и выводит конечный адрес после перенаправления.

12 Напишите программу, которая получает HTTP-ответ и выводит размер содержимого в байтах.

13 Напишите программу, которая выполняет GET-запрос и выводит все ключи заголовков ответа.

14 Напишите программу, которая отправляет GET-запрос с параметрами запроса (params) и выводит сформированный URL-адрес.

15 Напишите программу, которая получает HTTP-ответ и выводит количество заголовков, содержащихся в ответе сервера.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите, для чего предназначена библиотека Requests в языке программирования Python.

7.2 Перечислите, какие типы HTTP-запросов поддерживает библиотека Requests в языке программирования Python.

7.3 Опишите, как установить библиотеку Requests в языке программирования Python.

7.4 Объясните, что такое HTTP-код состояния ответа в языке программирования Python.

7.5 Опишите, как получить заголовки HTTP-ответа в языке программирования Python.

7.6 Сравните, GET- и POST-запросы в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.